

CRAFTING INTERPRETERS - ANSWER SHEET

Tyler Bacong

Chapters 10 and 11 end-of-chapter challenges

GitHub repository: <https://github.com/TylerBac0ng/crafting-interpreters-ch10-11>

The repository holds the complete jlox interpreter through Chapter 11 (functions, closures and the resolver), built on my Chapter 8 and 9 solutions. Build it with `javac -d build $(find src -name '*.java')` and run every test with `./run_tests.sh`. All 11 tests pass.

CHAPTER 10 - FUNCTIONS

Challenge 1 - Why Smalltalk does not check arity at runtime

Question: Our interpreter carefully checks that the number of arguments passed to a function matches the number of parameters it expects. Since this check is done at runtime on every call, it has a performance cost. Smalltalk implementations don't have that problem. Why not?

- In Smalltalk the number of arguments is part of the method's name (its *selector*). A unary message like `size` takes no arguments, a binary message like `+` takes exactly one, and a keyword message takes one argument per keyword: `at:put:` always takes two.
- So the parser already knows how many arguments a call passes just by reading the selector. `dict at: k put: v` sends `at:put:`, and any method named `at:put:` must declare exactly two parameters. A call and the method it finds can never disagree on the count.
- A call with the "wrong" number of arguments is really a call to a *different* selector, for example `at:` instead of `at:put:`. If the receiver has no method with that name, the normal method lookup fails and the object gets `doesNotUnderstand:`. There is no separate arity check: it is folded into the method lookup, which has to happen anyway.
- Lox is different because the call syntax `f(a, b)` does not say how many parameters `f` has, and `f` can be any value at runtime, so jlox must compare `arguments.size()` with `arity()` on every call.
- (Small caveat: Smalltalk blocks are values too, and `[ :x | x ] value: 1 value: 2` is checked when the block runs. But ordinary message sends, which are almost all calls, need no check.)

Challenge 2 - Anonymous functions

Question: Add anonymous function syntax to Lox so that `thrice(fun (a) { print a; });` works. How do you handle the tricky case of an anonymous function expression occurring in an expression statement: `fun () {};`

- Grammar: a new primary expression. A named declaration reuses the same rule after its name.

```
primary      -> ... | "fun" functionBody ;
funDecl      -> "fun" IDENTIFIER functionBody ;
functionBody -> "(" parameters? ")" block ;
```

- AST: a new `Expr.Function(List<Token> params, List<Stmt> body)` node. A named function is now `Stmt.Function(Token name, Expr.Function function)`, so declarations and lambdas share the same parser, resolver and `LoxFunction` code. `LoxFunction` stores the name separately and it is `null` for a lambda, which prints as `<fn>`.

- Parser: `functionBody()` parses the parameter list and body, and is called from both `function()` (declarations) and `primary()` (lambdas).

```
// Parser.java
private Stmt.Function function(String kind) {
    Token name = consume(IDENTIFIER, "Expect " + kind + " name.");
    return new Stmt.Function(name, functionBody(kind));
}

private Expr.Function functionBody(String kind) {
    consume(LEFT_PAREN, "Expect '(' after " + kind + " name.");
    ... parse parameters ...
    consume(LEFT_BRACE, "Expect '{' before " + kind + " body.");
    int enclosingLoopDepth = loopDepth;    // "break" can't escape a function
    loopDepth = 0;
    try {
        return new Expr.Function(parameters, block());
    } finally {
        loopDepth = enclosingLoopDepth;
    }
}

private Expr primary() {
    ...
    if (match(FUN)) return functionBody("function");
    ...
}
```

- Interpreter: evaluating the expression creates a closure over the current environment.

```
public Object visitFunctionExpr(Expr.Function expr) {
    return new LoxFunction(null, expr, environment);
}
```

- **The tricky case.** `declaration()` used to see `fun` and always parse a declaration, so `fun () {};` failed with "Expect function name." I added one token of lookahead: `fun` followed by an identifier is a declaration. Anything else falls through to `statement()`, which parses it as an expression statement, and the lambda is parsed in `primary()`.

```
private Stmt declaration() {
    try {
        if (check(FUN) && checkNext(IDENTIFIER)) {
            advance();
            return function("function");
        }
        if (match(VAR)) return varDeclaration();
        return statement();
    } ...
}
```

- This makes `fun () {};` a legal (if useless) statement that creates a function and discards it, the same as `1 + 2;`. It also allows a lambda to be called right away: `fun (a, b) { print a + b; }(20, 22);`. The other option would be to report a syntax error for a lambda at the start of a statement, like JavaScript does, but the lookahead is simple and never misreads a valid program.
- A related fix: `functionBody()` resets the loop depth from the break challenge, so a `break` in a function inside a loop is a syntax error instead of jumping out of the function (`tests/ch10_break_in_function.lox`).

Condensed from tests/ch10_lambda.lox (the output is shown in the comments):

```
thrice(fun (a) { print a; });           // 1 2 3
var square = fun (x) { return x * x; };
print square(7);                       // 49
fun makeAdder(n) { return fun (x) { return x + n; }; }
print makeAdder(10)(5);                // 15
fun () {};                             // legal, does nothing
fun (a, b) { print a + b; }(20, 22);    // 42
print square;                          // <fn>
```

It also works in the REPL from Chapter 8: typing `fun (x) { return x * 2; }(21)` prints 42.

Challenge 3 - Are parameters in the same scope as locals?

Question: Is this program valid? `fun scope(a) { var a = "local"; }` In other words, are a function's parameters in the same scope as its local variables, or in an outer scope? What does Lox do? What about other languages you are familiar with? What do you think a language should do?

- **Same scope.** `LoxFunction.call()` creates one new environment, defines the parameters in it, and runs the body's statements directly in that environment. The body is a `List<Stmt>`, not a `Stmt.Block`, so it does not get an extra environment of its own.
- **With the Chapter 10 interpreter, the program is valid.** `var a = "local";` calls `define()`, which just puts a new value into the map, so it silently replaces the parameter, the same way redeclaring a global works.
- **With the Chapter 11 resolver, it is an error.** `resolveFunction()` opens a single scope for the parameters and the body, and declaring a name twice in one local scope is reported. My interpreter gives:

```
[line 3] Error at 'a': Already a variable with this name in this scope.
```

(tests/ch10_param_scope.lox)

- Other languages (C, Java, Go and JavaScript tested on my machine):
 - * **C:** parameters are in the outermost block of the function. Compile error: redefinition of 'a'.
 - * **Java:** compile error: variable a is already defined in method.
 - * **Go:** compile error: a redeclared in this block.
 - * **JavaScript:** with `var` it is allowed and is the same variable (the redeclaration does nothing, it just assigns). With `let` it is a `SyntaxError` (Safari's engine: "Cannot declare a let variable twice: 'a'").
 - * **Python:** there are no declarations, so `a = "local"` simply rebinds the parameter.
 - * **Rust:** `let a = ...` is allowed and shadows the parameter, because shadowing is a normal, intended feature there.
- **What a language should do:** put parameters in the same scope as the body's top-level locals, and make redeclaring one an error. Once `var a` hides the parameter `a`, the argument the caller passed can never be read again, so the line is almost always a mistake. If parameters were in an outer scope, the code would quietly shadow them instead. This is what C, Java, Go and Lox (with the resolver) do. Rust is a reasonable exception, because its bindings are immutable by default and re-binding a name to a new value is a common idiom there.

CHAPTER 11 - RESOLVING AND BINDING

Challenge 1 - Why a function's name can be defined eagerly

Question: Why is it safe to eagerly define the variable bound to a function's name when other variables must wait until after they are initialized before they can be used?

- A variable has to wait because its initializer runs *right away*, during the declaration. In `var a = a;` the initializer would read `a` before it has any value, which is why the resolver reports "Can't read local variable in its own initializer."
- A function declaration has no initializer that runs. Its "value" is the function object, and creating it does not execute the body: the body is only stored in a closure. The body runs later, when the function is called, and by then the declaration has finished and the name is bound.
- In `jlox`, `visitFunctionStmt()` creates the `LoxFunction` and defines the name in one step. There is no point in between where the name exists but has no value, so code can never see a half-made function.
- This is also what makes recursion work: the body of `fib` refers to `fib`, and when the resolver walks the body it must already know that name, or the call would resolve to some outer variable (or a global) by mistake.
- The same reasoning covers anonymous functions in my 10.2 solution. In `var g = fun (n) { ... return g(n - 1); }`; the use of `g` is inside the lambda's body, which is not run during the initializer. The resolver only checks the innermost scope for "own initializer" reads, so this is allowed, and it works when called (I tested it: a recursive local lambda returns the right answer).

Challenge 2 - `var a = a;` in other languages

Question: How do other languages you know handle local variables that refer to the same name in their initializer, like `var a = "outer"; { var a = a; }` Is it a runtime error? Compile error? Allowed? Do they treat global variables differently? Do you agree with their choices? Justify your answer.

I tested each of these on my machine (Rust and Scheme from their documentation):

Language	Local <code>var a = a;</code>	Global
Lox (ch. 11)	Compile error (resolver): "Can't read local variable in its own initializer."	Allowed: reads the old global. Undefined name -> runtime error.
C	Allowed. Reads the new, uninitialized <code>a</code> . clang warns; the program printed garbage (214536064).	File-scope <code>int a = a;</code> is a compile error (not a constant).
Java	Compile error. A local can't shadow a local ("already defined"), and shadowing a field gives "might not have been initialized".	<code>static int a = a;</code> is a compile error: "self-reference in initializer".
JavaScript	<code>let</code> : runtime <code>ReferenceError</code> (temporal dead zone). <code>var</code> : same variable, keeps "outer".	Same rules as locals.
Python	<code>Runtime UnboundLocalError</code> (<code>a</code> is local to the whole function).	Module level: same variable, so <code>a = a</code> just re-assigns it.
Go	<code>a := a</code> is allowed and reads the outer <code>a</code> (printed "outer").	<code>var g = g</code> is a compile error: "initialization cycle".
Ruby	Inside a method: new local, value <code>nil</code> . In a block: same outer variable.	-

Language	Local <code>var a = a;</code>	Global
Rust	<code>let a = a;</code> is allowed, reads the outer <code>a</code> (normal shadowing).	-
Scheme	<code>let:</code> reads the outer <code>a</code> . <code>letrec:</code> an error (<code>a</code> has no value yet).	<code>define</code> at top level is allowed.

- **Do globals get treated differently?** Often, yes. Lox, Python and Go all handle globals differently from locals, mostly because globals can be referenced before their declaration (from function bodies, or later REPL lines), so the compiler can't always tell what order things will happen in. Go goes the other way and checks globals more strictly, rejecting the initialization cycle at compile time.
- **Do I agree?** For locals, I agree with Lox and Java: make it a compile-time error. Reasons:
 - * The line uses one name for two different variables, and readers can't tell which one is meant. The languages above don't even agree: C reads the new (garbage) value, Go and Rust read the outer one, JavaScript and Python throw at runtime.
 - * C's behavior is the worst choice: no error, just undefined behavior. A runtime error (JavaScript, Python) is better but only shows up if that line happens to run.
 - * A compile error costs the user almost nothing: they rename the variable. It is also the conservative choice, since a language can relax an error later without breaking programs, but can't add one later.
- For globals, I think Lox's choice is acceptable. Top-level code is often typed into a REPL one line at a time, and `var a = a;` there just means "redefine `a` from its old value", which is harmless. Reading a global that was never defined is still caught (as a runtime error).

Challenge 3 - Report unused local variables

Question: Extend the resolver to report an error if a local variable is never used.

- The book's resolver stores a `Boolean` per variable. I replaced it with a small `Variable` class that also records a `used` flag (and the slot index for challenge 4). Scopes are `LinkedHashMaps` so errors come out in source order.
- `resolveLocal()` sets `used` when a variable expression *reads* the variable. When a scope ends, `endScope()` reports every variable that was never read.

```
private static class Variable {
    final Token name;
    final int index;           // challenge 4
    boolean defined = false;
    boolean used = false;     // challenge 3
    ...
}
```

```
private void endScope() {
    Map<String, Variable> scope = scopes.pop();
    for (Variable variable : scope.values()) {
        if (!variable.used) {
            Lox.error(variable.name, "Local variable '" +
                variable.name.lexeme + "' is never used.");
        }
    }
}
```

```
private void resolveLocal(Expr expr, Token name, boolean isRead) {
    for (int i = scopes.size() - 1; i >= 0; i--) {
        Variable variable = scopes.get(i).get(name.lexeme);
        if (variable != null) {
            interpreter.resolve(expr, scopes.size() - 1 - i, variable.index);
            if (isRead) variable.used = true;
            return;
        }
    }
}
```

- Design choices:

- * **Only reads count.** `visitAssignExpr` passes `isRead = false`. A variable that is only assigned to is still useless, because its value is never looked at.
- * **Parameters are not reported.** A callback often has to accept arguments it doesn't need (for example the `fn` passed to `thrice`), so this would give false alarms.
- * **Globals are not reported.** They are not tracked by the resolver, and code in a later file or REPL line may use them.
- * A read from inside a closure counts, since `resolveLocal` walks out through all the enclosing scopes. Local functions count too: an unused local `fun` is reported.
- * The challenge asks for an error, so it sets `hadError` and the program does not run. In a real language I would make this a warning instead.

From `tests/ch11_unused.lox` (the errors it produces are shown in the comments):

```
fun f(unusedParam) {           // parameter: not reported
    var used = 1;
    var unused = 2;             // [line 6] Error at 'unused': ... is never used.
    var onlyAssigned;
    onlyAssigned = 3;           // [line 7] Error at 'onlyAssigned': ... is never used.
    fun helper() {}             // [line 9] Error at 'helper': ... is never used.
    print used;
}
{
    var outer = "read inside a closure counts"; // not reported
    fun show() { print outer; }
    show();
}
```

Challenge 4 - Store local variables in an array

Question: Extend the resolver to associate a unique index for each local variable declared in a scope. When resolving a variable access, look up both the scope the variable is in and its index and store that. In the interpreter, use that to quickly access a variable by its index instead of using a map.

- **Resolver:** when a local is declared, its index is the number of variables already declared in that scope (`scope.size()`). `resolveLocal()` (above) sends both the depth and the index to the interpreter.

```
private void declare(Token name) {
    if (scopes.isEmpty()) return;
    Map<String, Variable> scope = scopes.peek();
    if (scope.containsKey(name.lexeme)) {
        Lox.error(name, "Already a variable with this name in this scope.");
        return;
    }
    scope.put(name.lexeme, new Variable(name, scope.size()));
}
```

- **Environment:** a local scope now stores values in an `ArrayList` instead of a `HashMap`. `define()` appends; `getAt` and `assignAt` walk out distance scopes and index into the array. No names are looked up at all.

```
class Environment {
    final Environment enclosing;
    private final List<Object> values = new ArrayList<>();

    void define(Object value) { values.add(value); }

    Object getAt(int distance, int index) {
        return ancestor(distance).values.get(index);
    }

    void assignAt(int distance, int index, Object value) {
        ancestor(distance).values.set(index, value);
    }
}
```

- **Interpreter:** locals maps each variable/assign expression to a `Slot` (depth, index). Globals stay in a name-to-value map, because a function body can refer to a global that is declared later in the file, so its position isn't known when the body is resolved.

```
private Object lookUpVariable(Token name, Expr expr) {
    Slot slot = locals.get(expr);
    if (slot != null) {
        return environment.getAt(slot.depth, slot.index);
    }
    if (globals.containsKey(name.lexeme)) {
        return globals.get(name.lexeme);
    }
    throw new RuntimeError(name, "Undefined variable '" + name.lexeme + "'.");
}

private void declare(Token name, Object value) { // used by var and fun
    if (environment == globalEnvironment) {
        globals.put(name.lexeme, value);
    } else {
        environment.define(value); // lands at the resolver's index
    }
}
```

- **Why the indexes line up:** the resolver numbers a scope's variables in the order they are declared in the source, and at runtime they are defined in that same order, because statements in a block run top to bottom. A declaration can't be skipped while a later one in the same scope runs: the grammar does not allow a bare `var` as the body of an `if` or `while`, so that always needs its own block (its own scope).

`LoxFunction.call()` defines the parameters first, so they take slots 0 to $n-1$, matching `resolveFunction()`.

- Test (`tests/ch11_slots.lox`) covers several locals in a scope, parameters plus locals, recursion, closures that read and write captured variables, the closure bug example from the chapter (prints `global` twice), shadowing, and `break/return` in loops in functions. All output matches.
- **Speed:** `benchmark/locals.lox` runs a 5,000,000-iteration loop over block locals. It took about 2.2 s on my Chapter 8/9 interpreter (locals in a `HashMap`) and about 1.4 s on this one (array slots), including JVM startup, so roughly 35-40% faster. That is a rough comparison since the two interpreters differ in other small ways.

All 11 tests pass with `./run_tests.sh`.